ESO

July 2, 2025

[2]:
```
''' SPECIFICATION OF DATA set :
1.levelTo - levelFrom in BOAs.csv likely represents the change in battery␣
 ↪behavior over each interval
2.Data Nature: Likely time-series, possibly with multiple influencing variables
3. Is a trainning source boa.csv

Best Choosen algorithim is lstm :Best for non-linear battery behavior,␣
 ↪sequences; handles long-term dependencies
Our data set is multivariate i-e there are many indepent variable that␣
 ↪influence target result, so prophet or Sarima is not suitable.
We will predict and forcast on well parameters with perfect accuracy through ␣
 ↪lstm.....'''
```

[2]:
```
' SPECIFICATION OF DATA set :\n1.levelTo - levelFrom in BOAs.csv likely
represents the change in battery behavior over each interval'
```

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score, mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# === STEP 1: Load Data ===
df = pd.read_csv("BOAs.csv")
df['timeFrom'] = pd.to_datetime(df['timeFrom'])
df.sort_values('timeFrom', inplace=True)
df['battery_output'] = df['levelTo'] - df['levelFrom']

# === STEP 2: Feature Engineering ===
df['hour'] = df['timeFrom'].dt.hour
df['day_of_week'] = df['timeFrom'].dt.dayofweek

# Select features
```

```
features = ['levelFrom', 'levelTo', 'soFlag', 'storFlag', 'rrFlag', 'hour',␣
 ↪'day_of_week']
df = df[['timeFrom', 'battery_output'] + features]
df.fillna(0, inplace=True)
df.set_index('timeFrom', inplace=True)

# === STEP 3: Resample to hourly frequency ===
df_resampled = df.resample('1h').mean().fillna(0)

# === STEP 4: Scale all features ===
scalers = {}
scaled_df = pd.DataFrame(index=df_resampled.index)

for col in df_resampled.columns:
    scaler = MinMaxScaler()
    scaled_df[col] = scaler.fit_transform(df_resampled[[col]])
    scalers[col] = scaler

# === STEP 5: Create Sequences ===
def create_multivariate_sequences(data, target_col, window=48):
    X, y = [], []
    for i in range(window, len(data)):
        X.append(data.iloc[i-window:i].values)
        y.append(data.iloc[i][target_col])
    return np.array(X), np.array(y)

X, y = create_multivariate_sequences(scaled_df, 'battery_output', window=48)
split = int(len(X) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# === STEP 6: Build LSTM ===
model = Sequential([
    Input(shape=(X_train.shape[1], X_train.shape[2])),
    LSTM(128, return_sequences=True),
    Dropout(0.3),
    LSTM(64),
    Dropout(0.3),
    Dense(1)
])
model.compile(optimizer='adam', loss='mse')
early_stop = EarlyStopping(monitor='val_loss', patience=10,␣
 ↪restore_best_weights=True)

model.fit(X_train, y_train,
          validation_data=(X_test, y_test),
          epochs=100,
```

```
          batch_size=32,
          callbacks=[early_stop],
          verbose=1)

# === STEP 7: Evaluate ===
y_pred = model.predict(X_test)
y_pred_inv = scalers['battery_output'].inverse_transform(y_pred)
y_test_inv = scalers['battery_output'].inverse_transform(y_test.reshape(-1, 1))

r2 = r2_score(y_test_inv, y_pred_inv)
rmse = np.sqrt(mean_squared_error(y_test_inv, y_pred_inv))
mse = mean_squared_error(y_test_inv, y_pred_inv)

print("\n Evaluation:")
print(f"R² Score: {r2:.4f}")
print(f"RMSE    : {rmse:.4f}")
print(f"MSE     : {mse:.4f}")

# === STEP 8: Forecast Next 10 Days (240 hours) ===
future_steps = 240
last_seq = scaled_df[-48:].values
future_preds = []

for _ in range(future_steps):
    pred = model.predict(last_seq.reshape(1, 48, X.shape[2]), verbose=0)[0][0]
    future_preds.append(pred)
    new_row = np.append(last_seq[1:], [[*last_seq[-1][:-1], pred]], axis=0)
    last_seq = new_row

future_output = scalers['battery_output'].inverse_transform(np.
 ↪array(future_preds).reshape(-1, 1))

# === STEP 9: Plot Results ===
plt.figure(figsize=(20, 8))

plt.subplot(1, 2, 1)
plt.plot(y_test_inv, label="Actual", color="royalblue")
plt.plot(y_pred_inv, label="Predicted", color="darkorange")
plt.title("Test Set: Actual vs Predicted", fontsize=14)
plt.xlabel("Time Step")
plt.ylabel("Battery Output")
plt.grid(True)
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(future_output, label="Forecast (Next 10 Days)", color="green")
plt.title("Forecasted Battery Output", fontsize=14)
```

```
plt.xlabel("Hour")
plt.ylabel("Predicted Output")
plt.grid(True)
plt.legend()

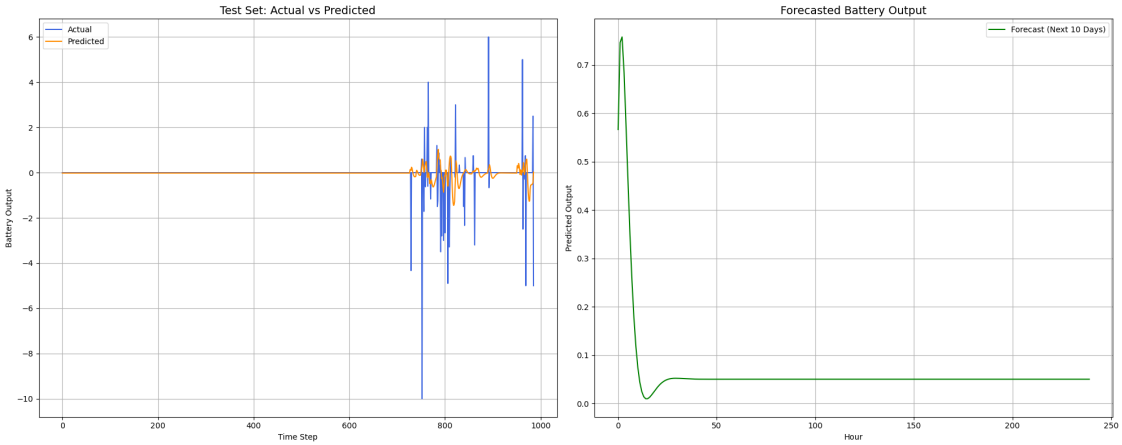
plt.tight_layout()
plt.show()
```

```
Epoch 1/100
124/124             9s 51ms/step -
loss: 0.0158 - val_loss: 0.0011
Epoch 2/100
124/124             9s 37ms/step -
loss: 0.0016 - val_loss: 0.0014
Epoch 3/100
124/124             6s 47ms/step -
loss: 0.0015 - val_loss: 0.0015
Epoch 4/100
124/124             6s 46ms/step -
loss: 0.0013 - val_loss: 0.0019
Epoch 5/100
124/124             6s 45ms/step -
loss: 0.0013 - val_loss: 0.0012
Epoch 6/100
124/124             6s 46ms/step -
loss: 0.0010 - val_loss: 0.0013
Epoch 7/100
124/124             5s 44ms/step -
loss: 0.0010 - val_loss: 0.0013
Epoch 8/100
124/124             10s 38ms/step -
loss: 9.3277e-04 - val_loss: 0.0014
Epoch 9/100
124/124             7s 50ms/step -
loss: 8.6564e-04 - val_loss: 0.0011
Epoch 10/100
124/124             9s 41ms/step -
loss: 7.4886e-04 - val_loss: 0.0011
Epoch 11/100
124/124             6s 46ms/step -
loss: 6.5003e-04 - val_loss: 9.8916e-04
Epoch 12/100
124/124             10s 44ms/step -
loss: 9.7777e-04 - val_loss: 8.8886e-04
Epoch 13/100
124/124             6s 48ms/step -
loss: 5.2867e-04 - val_loss: 8.9575e-04
Epoch 14/100
```

```
124/124             15s 88ms/step -
loss: 4.6772e-04 - val_loss: 8.5125e-04
Epoch 15/100
124/124             24s 112ms/step -
loss: 4.2070e-04 - val_loss: 8.9352e-04
Epoch 16/100
124/124             20s 107ms/step -
loss: 3.9092e-04 - val_loss: 8.9109e-04
Epoch 17/100
124/124             15s 60ms/step -
loss: 4.0221e-04 - val_loss: 8.5686e-04
Epoch 18/100
124/124             11s 63ms/step -
loss: 3.8162e-04 - val_loss: 8.3497e-04
Epoch 19/100
124/124             12s 98ms/step -
loss: 4.3521e-04 - val_loss: 8.0906e-04
Epoch 20/100
124/124             17s 70ms/step -
loss: 3.1784e-04 - val_loss: 8.2807e-04
Epoch 21/100
124/124             7s 56ms/step -
loss: 3.5466e-04 - val_loss: 8.1758e-04
Epoch 22/100
124/124             8s 62ms/step -
loss: 2.0749e-04 - val_loss: 7.7462e-04
Epoch 23/100
124/124             9s 54ms/step -
loss: 2.0208e-04 - val_loss: 8.7012e-04
Epoch 24/100
124/124             7s 57ms/step -
loss: 1.5956e-04 - val_loss: 7.9575e-04
Epoch 25/100
124/124             10s 55ms/step -
loss: 2.9384e-04 - val_loss: 7.9166e-04
Epoch 26/100
124/124             6s 50ms/step -
loss: 2.7632e-04 - val_loss: 7.6890e-04
Epoch 27/100
124/124             11s 58ms/step -
loss: 1.0026e-04 - val_loss: 7.7097e-04
Epoch 28/100
124/124             6s 50ms/step -
loss: 1.3008e-04 - val_loss: 8.6869e-04
Epoch 29/100
124/124             13s 69ms/step -
loss: 1.7310e-04 - val_loss: 7.8401e-04
Epoch 30/100
```

```
124/124             11s 73ms/step -
loss: 2.6855e-04 - val_loss: 7.5971e-04
Epoch 31/100
124/124             7s 59ms/step -
loss: 1.0627e-04 - val_loss: 8.4652e-04
Epoch 32/100
124/124             9s 50ms/step -
loss: 1.2176e-04 - val_loss: 8.0159e-04
Epoch 33/100
124/124             6s 50ms/step -
loss: 3.9037e-04 - val_loss: 7.4662e-04
Epoch 34/100
124/124             10s 46ms/step -
loss: 1.2695e-04 - val_loss: 7.7241e-04
Epoch 35/100
124/124             8s 61ms/step -
loss: 2.2731e-04 - val_loss: 8.3088e-04
Epoch 36/100
124/124             9s 51ms/step -
loss: 1.4693e-04 - val_loss: 8.0461e-04
Epoch 37/100
124/124             6s 50ms/step -
loss: 8.1050e-05 - val_loss: 7.8515e-04
Epoch 38/100
124/124             7s 53ms/step -
loss: 9.3795e-05 - val_loss: 7.5889e-04
Epoch 39/100
124/124             6s 48ms/step -
loss: 1.6821e-04 - val_loss: 7.9598e-04
Epoch 40/100
124/124             6s 50ms/step -
loss: 1.1983e-04 - val_loss: 8.0920e-04
Epoch 41/100
124/124             10s 48ms/step -
loss: 1.7723e-04 - val_loss: 7.6780e-04
Epoch 42/100
124/124             7s 58ms/step -
loss: 2.7507e-04 - val_loss: 7.4842e-04
Epoch 43/100
124/124             7s 52ms/step -
loss: 2.2131e-04 - val_loss: 7.6690e-04
31/31            1s 22ms/step

 Evaluation:
R² Score: -0.1179
RMSE    : 0.6558
MSE     : 0.4301
```



[ ]:
```
" AS WE CAN SEE THE LOSS IN DATA AND WORST EVALUATION MATRICES THERE IS␣
↪SIGNIFICANCE THAT LSTM IS NOT ABLE TO HANDLE THE DATA SET AS ITS TOO NOISY␣
↪AND UNINFORM "
```